

Here's a clear evaluation of the machine learning results you shared for predicting car origin (1 = USA, 2 = Europe, 3 = Japan) based on technical specs.

Quick Summary Table

Model	Overall Accuracy	USA (1) F1	Europe (2) F1	Japan (3) F1	Macro F1	Class Imbalance Handling
Decision Tree	78%	0.87	0.63	0.71	0.74	decent
Random Forest	76%	0.87	0.63	0.59	0.70	slightly worse

Support (test set size): USA = 43, Europe = 20, Japan = 16 → **clear imbalance** (USA dominant)

Key Observations

- 1. Performance is quite similar
 - Decision Tree slightly edges out Random Forest (78% vs 76%)
 - This is unusual — Random Forest is normally more robust. Possible reasons:
 - Small dataset (~79 test samples → total dataset probably ~300–400 rows)
 - Single tree got lucky with a good split
 - Random Forest might be slightly over-regularized or suffers from class imbalance

2. Class-wise behavior (most important part)

Class	Precision	Recall	F1	Interpretation
USA (1)	~0.82–0.83	0.91–0.93	0.87	very reliable — high recall = catches most American cars
Europe (2)	0.62–0.73	~0.55–0.65	0.63	weakest and most inconsistent class
Japan (3)	~0.73	0.50–0.69	0.59–0.71	low recall in RF → misses many Japanese cars

→ The model is **good at recognizing American cars**, but **struggles to distinguish European vs Japanese cars** (especially in Random Forest).

3. Feature importance (Random Forest – most trustworthy here)

Rank	Feature	Importance	Interpretation
1	displacement	32.2%	by far the strongest signal
2	weight	18.2%	strongly correlated with displacement
3	mpg	15.6%	Japanese cars typically much higher mpg
4	horsepower	15.3%	European cars often high hp for their size
5	acceleration	10.9%	related to power-to-weight
6	cylinders	7.7%	useful but weaker (many overlaps)

→ The model has mostly learned "**bigger engine + heavier = American**", which is historically very reasonable (especially 1970–1982 era data, typical for this dataset).

Main Problems / Limitations

- **Small test set** (79 samples) → results have high variance. One or two different cars in the test split can move accuracy by 3–5%.
- **Class imbalance** not handled → no `class_weight='balanced'`, no SMOTE/oversampling visible.
- **Europe vs Japan confusion** — very typical in this classic cars dataset (Auto MPG /

UCI). Many European and Japanese models from the 70s/80s had similar displacement/weight/mpg ranges.

- No signs of hyperparameter tuning (max_depth, min_samples_leaf, etc.).
- No cross-validation reported → single train/test split is noisy.

Realistic Verdict

78% accuracy is actually decent for this problem and dataset size, especially considering:

- 3 classes
- strong class imbalance
- very small test set
- overlapping feature distributions between Europe & Japan

This is **not a bad baseline**, but also **far from production-ready**.

Quick Recommendations to Improve (in order of expected impact)

1. Use **cross-validation** (5- or 10-fold) instead of single split — get more stable estimate.
2. Try **class_weight='balanced'** in RandomForest / DecisionTree or use class_weight dict.
3. Add **stratified split** (already good practice, but worth confirming).
4. Try **LightGBM / XGBoost** with scale_pos_weight or is_unbalance parameter — usually stronger on small & imbalanced data.
5. Feature engineering ideas:
 - displacement per cylinder
 - power-to-weight ratio (hp / weight)
 - mpg × weight (proxy for efficiency under load)
6. Consider **confusion matrix visualization** — you'll almost certainly see most errors

between class 2 $\leftrightarrow$ 3.

7. If dataset is the classic Auto MPG UCI dataset $\rightarrow$ many people reach **82–88%** with tuned gradient boosting + the tricks above.

Let me know if you have the confusion matrix, the dataset size, or want suggestions for next concrete steps (code snippets welcome). Overall: solid first attempt, clear path to ~8–12% improvement still exists.